

Informe Técnico Final

Editor Interactivo de Imágenes de Frutas

Carrera: Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial

Instituto: IFTS N°18 – CABA

Materia: Técnicas de Procesamiento Digital de Imágenes

Profesor: Bonini, Juan Ignacio

Ciclo: 2026

Alumnos: Armiento, Fernando – Caviglia, Paula

1. Problema

Las técnicas de procesamiento digital de imágenes involucran conceptos matemáticos que resultan difíciles de comprender en abstracto: la varianza inter-clase de Otsu, el rol del radio en un filtro pasa bajos de Fourier, o cómo un cambio en el canal H del espacio HSV modifica el color percibido de una fruta. Sin una herramienta que permita experimentar en tiempo real, el aprendizaje de estos conceptos queda limitado a fórmulas y demostraciones estáticas.

El problema que resolvemos es concreto. Necesitábamos una herramienta interactiva que permitiera aplicar y comparar distintas técnicas de PDI sobre imágenes reales de frutas, ajustando los parámetros de cada técnica y observando el efecto de manera inmediata. La fruta como objeto de estudio resulta apropiada porque sus características visuales (color saturado, bordes bien definidos, fondo generalmente uniforme) hacen que las diferencias entre técnicas sean fáciles de percibir incluso con imágenes pequeñas.

El mismo concepto tiene aplicaciones concretas. Podría ser utilizado para realizar una clasificación automática de frutas en líneas de producción, detección de madurez mediante análisis de color en cosechas por drones, o segmentación de objetos de interés en imágenes de robots manipuladores en ambientes agrícolas.

2. Enfoque

Optamos por construir una **herramienta interactiva de edición**, no un pipeline de procesamiento automático. La diferencia recae en que el usuario elige la técnica y controla sus parámetros con sliders, y la aplicación responde en tiempo real. Esto convierte la herramienta en un entorno de exploración antes que en un sistema de análisis automatizado.

El foco es la edición, la visualización y la comprensión de cada técnica por separado.

Para la arquitectura del código, elegimos el patrón de fachada con tres capas bien diferenciadas:

- Una capa de **procesamiento básico** (Pillow), para operaciones que no requieren visión por computadora.
- Una capa de **procesamiento avanzado** (OpenCV), que concentra todas las técnicas de PDI del programa de la materia.
- Una capa de **presentación** (Motor + interfaces), completamente separada del procesamiento.

Esta separación permite reutilizar el mismo core desde dos interfaces distintas sin duplicar ni una sola línea de lógica de procesamiento. Cualquier corrección o mejora en una técnica se refleja automáticamente en ambas interfaces.

3. Técnicas utilizadas

3.1 Filtro gaussiano, brillo y contraste

El filtro gaussiano suaviza la imagen convolucionando cada píxel con sus vecinos usando pesos distribuidos según una gaussiana. Su efecto principal es eliminar variaciones de alta frecuencia (ruido) que de otra manera generarían falsos bordes en las etapas siguientes. Lo implementamos con Pillow (`GaussianBlur`) para aprovechar la precisión subpíxel de su implementación.

El ajuste de brillo y contraste se implementa con Pillow (`ImageEnhance.Brightness` y `ImageEnhance.Contrast`), que trabaja con factores multiplicativos. Como los sliders del proyecto usan un modelo aditivo (−100 a +100), Motor realiza la conversión $\text{factor} = (128 + \text{offset}) / 128$, de modo que un offset de 0 produce factor 1.0 (sin cambio), +100 produce ~1.78 y −100 produce ~0.22.

Parámetros: kernel de blur (0–10, mapeado a tamaño impar 1–21 px), offset de brillo (−100 a +100), factor de contraste (0.1x a 3.0x).

3.2 Detección de bordes con Canny

El algoritmo de Canny (1986) es el estándar para detección de bordes por su comportamiento óptimo en tres criterios simultáneos: detección correcta (no omite bordes reales ni genera falsos), localización precisa (el borde detectado coincide con el borde real) y respuesta única (un solo borde por discontinuidad real).

Su pipeline tiene cuatro etapas: (1) suavizado gaussiano con kernel 5x5 para eliminar ruido, (2) cálculo del gradiente de Sobel en X e Y para obtener magnitud y dirección, (3) supresión de no-máximos que adelgaza los bordes a un píxel de ancho, y (4) histéresis de doble umbral que conserva solo los bordes conectados a zonas de gradiente alto.

Parámetros: umbral bajo (0–300, default 40) y umbral alto (0–300, default 120). La relación recomendada entre ambos es 1:2 o 1:3. En el resultado, los bordes se muestran en verde sobre fondo negro.

3.3 Segmentación con K-means en espacio HSV

K-means es un algoritmo de clustering no supervisado que agrupa los píxeles en k grupos según su similitud de color, minimizando la suma de distancias cuadráticas de cada píxel al centroide de su cluster. Elegimos operar en espacio HSV en lugar de BGR porque el canal H (tono) codifica el color de forma perceptualmente uniforme e independiente de la iluminación, lo que produce clusters más coherentes con la percepción humana.

El proceso aplana la imagen de $H \times W \times 3$ a una lista de píxeles ($N \times 3$), ejecuta K-means con 10 reinicios aleatorios eligiendo el de menor distorsión total, y reconstruye la imagen reemplazando cada píxel por el color del centroide de su cluster. Con $k=4$ sobre una imagen de manzana se obtienen típicamente cuatro regiones: cuerpo de la fruta, zona de sombra, hoja, y fondo.

Parámetros: número de clusters k (2–6, default 4). Criterio de convergencia: error < 0.2 o máximo 100 iteraciones por intento.

3.4 Umbralización con Otsu y umbral manual

El método de Otsu (1979) calcula automáticamente el umbral óptimo T que maximiza la varianza inter-clase entre fondo y objeto:

$$\text{var_b}(T) = w_0(T) w_1(T) [\mu_0(T) - \mu_1(T)]^2$$

donde w_0 y w_1 son las probabilidades de pertenencia a cada clase y μ_0 , μ_1 sus medias de intensidad. El método funciona mejor cuando el histograma de la imagen es bimodal, como ocurre típicamente en imágenes de frutas sobre fondo uniforme.

El modo también permite umbral manual: el usuario fija el valor T en lugar de calcularlo automáticamente. En ambos casos, el resultado se muestra como un overlay en el que la zona detectada como objeto conserva el color original y la zona de fondo se oscurece con un tinte azul con el contorno en amarillo.

Parámetros: umbral (0 = automático Otsu, 1–255 = manual). Preprocesamiento: blur gaussiano 5x5. Postprocesamiento: limpieza morfológica de la máscara.

3.5 Watershed

Watershed trata la imagen como un mapa topográfico donde los valores de intensidad representan alturas. El algoritmo "inunda" el terreno desde semillas (mínimos locales robustos) y construye bordes donde distintas inundaciones se encuentran. Es especialmente útil cuando varias frutas se tocan y técnicas más simples las detectarían como un único objeto.

Implementamos la variante con marcadores basada en la transformada de distancia. El pipeline completo tiene diez pasos: binarización con Otsu, apertura morfológica para limpiar ruido, dilatación para obtener "fondo seguro", transformada de distancia euclidiana (cada píxel blanco toma el valor de su distancia al borde más cercano), umbralización de esa transformada para obtener las "semillas" (núcleo de cada objeto), etiquetado de componentes conectadas, asignación de zona desconocida (la frontera entre objetos y fondo), aplicación del algoritmo, y coloreado de cada

segmento con un color distinto. Los bordes calculados por Watershed se pintan en rojo (etiqueta -1).

Parámetros: umbral de la transformada de distancia (1–90 % del máximo). Valores bajos producen semillas más grandes y menos separación; valores altos producen semillas pequeñas y más separación de objetos.

3.6 Histograma con ecualización

El histograma de una imagen es la distribución de frecuencias de los niveles de intensidad (0–255). Un histograma concentrado en valores bajos indica imagen oscura; concentrado en valores altos, imagen clara; extendido uniformemente, buen contraste.

La ecualización aplica la transformación $s = (L-1) \cdot \text{CDF}(r)$, donde CDF es la función de distribución acumulada del histograma original. El resultado redistribuye los píxeles de forma que el histograma resultante sea aproximadamente uniforme, ampliando el rango dinámico y revelando detalles en zonas sobreexpuestas o subexpuestas.

Implementamos la visualización con dos curvas superpuestas: la distribución original (color tenue) y la ecualizada (color brillante del canal), sobre un lienzo dibujado directamente con OpenCV para poder mostrarlo sin depender de matplotlib en tiempo real.

Parámetros: canal a visualizar (Grisos / R / G / B). El selector usa radio buttons en la interfaz web y trackbar de valores discretos en la interfaz de escritorio.

3.7 Transformada de Fourier y filtro pasa bajos

La Transformada Discreta de Fourier (DFT) descompone la imagen en sus componentes de frecuencia. En el espectro centrado, las bajas frecuencias (formas globales, cambios suaves) se concentran en el centro; las altas frecuencias (bordes, texturas finas, ruido) aparecen en la periferia.

Implementamos dos modos bajo el mismo slider. Con radio=0 se muestra el espectro: se calcula $\log(1 + |DFT|)$ normalizado y se aplica el colormap INFERNO de OpenCV para hacer visibles las diferencias de magnitud entre frecuencias. Con radio>0 se aplica un filtro pasa bajos circular: se crea una máscara binaria que anula todas las frecuencias fuera del radio indicado, se aplica al espectro centrado, y se reconstruye la imagen con IDFT. El resultado muestra la imagen sin las texturas finas, preservando solo los contornos generales.

Parámetros: radio del filtro en píxeles (0 = mostrar espectro, 1–100 = filtrar). La transición entre ambos modos ocurre en el mismo slider.

3.8 Ajuste de tono, saturación y brillo en HSV

La manipulación directa en espacio HSV permite cambiar el color de la fruta (canal H), la intensidad del color (canal S) y la luminosidad (canal V) de forma independiente, algo que no es posible operando directamente en BGR.

La implementación convierte a HSV con precisión float32, aplica $H = (H + \text{delta}_h) \% 180$ (el módulo maneja el wraparound del rojo), escala S y V con clip a [0, 255], y convierte de vuelta a BGR. El canal H en OpenCV usa el rango 0–180 (no 0–360), por lo que el desplazamiento máximo de ± 90 equivale a media rueda de colores.

Parámetros: desplazamiento de H (–90 a +90), factor de saturación (0–200 %, donde 100 % = sin cambio), factor de brillo (0–200 %, donde 100 % = sin cambio).

4. Implementación

4.1 Estructura del proyecto

```
TP FINAL/
  main.py          -- Interfaz escritorio: OpenCV, sliders en tiempo real
  app.py           -- Interfaz web: Streamlit, carga de imagen desde disco
  motor.py         -- Fachada del core (única clase que importan las interfaces)
  pillow_processor.py -- PillowProcessor: operaciones básicas con Pillow
  opencv_processor.py -- OpenCVProcessor: técnicas avanzadas con OpenCV
  bordes.py        -- Canny, Sobel, blur gaussiano
  color.py         -- Conversión BGR-HSV, ajustes de color
  histograma.py    -- Cálculo, ecualización y visualización de histogramas
  morfologia.py    -- Erosion, dilatación, apertura, cierre
  segmentacion_kmeans.py -- K-means en BGR y HSV
  segmentacion_otsu.py -- Umbralización con Otsu y overlay
  watershed.py     -- Watershed con marcadores basado en transformada de distancia
  requirements.txt  -- opencv-python, numpy, matplotlib, scikit-learn, Pillow, streamlit
  images/          -- Dataset de frutas (no incluido en el repositorio)
```

4.2 Las tres clases del core

PillowProcessor

Recibe un array numpy BGR en el constructor, lo convierte a RGB y lo almacena como PIL.Image.

Todos los métodos retornan self para permitir encadenamiento

(proc.aplicar_blur(2).ajustar_brillo(1.2).ajustar_contraste(1.5)). El método obtener_imagen() convierte de vuelta a numpy BGR.

```
Constructor: imagen_bgr (BGR) → convierte a RGB → PIL.Image interno
aplicar_blur(radio)    -- GaussianBlur de Pillow (radio en píxeles flotantes)
ajustar_brillo(factor) -- ImageEnhance.Brightness (1.0 = sin cambio)
ajustar_contraste(factor) -- ImageEnhance.Contrast (1.0 = sin cambio)
convertir_grises()    -- ImageOps.grayscale + convert("RGB")
obtener_imagen()      -- convierte PIL.Image (RGB) → numpy (BGR)
```

OpenCVProcessor

Recibe un array numpy BGR y lo almacena sin conversión. Cada método delega en las funciones de los módulos existentes, sin reescribir lógica.

```

Constructor: imagen_bgr (BGR) → almacena referencia
bordes_canny(bajo, alto) → delega en bordes.canny_coloreado()
segmentar_kmeans(k) → delega en segmentacion_kmeans.segmentar_kmeans_hsv()
umbralizar_otsu() → delega en segmentacion_otsu.otsu_con_overlay()
umbralizar_manual(umbral) → lógica inline (threshold manual + overlay + contornos)
aplicar_watershed(umbral_dist) → delega en watershed.aplicar_watershed()
histograma_canal(canal, h, w) → delega en histograma.dibujar_histograma_canal()
espectro_fourier() → DFT + log(1+magnitud) + COLORMAP_INFERNO
filtrar_fourier(radio) → DFT + máscara circular + IDFT
ajustar_hsv(delta_h, s, v) → delega en color.ajustar_hsv()

```

Motor

Es la fachada. Recibe el array BGR, crea el procesador apropiado para cada operación y expone un método por transformación. Es la única clase que las interfaces importan.

```

from motor import Motor

motor = Motor(imagen_bgr)
resultado = motor.blur_y_brillo(kernel=5, brillo=20, contraste=1.2)

```

El método `blur_y_brillo` convierte el parámetro de brillo antes de delegar:

```

radio = max(0.0, (kernel_size - 1) / 2.0)
factor_brillo = max(0.01, (128.0 + brillo) / 128.0) # offset → factor Pillow

```

El método `umbralizar_otsu` elige el procesador según el valor del parámetro:

```

if umbral_manual == 0:
    return proc.umbralizar_otsu() # Otsu automático
return proc.umbralizar_manual(umbral) # umbral fijo

```

El método `fourier` elige entre espectro y filtro:

```

if radio == 0:
    return proc.espectro_fourier() # visualiza espectro
return proc.filtrar_fourier(radio) # aplica pasa bajos y reconstruye

```

4.3 Flujo de datos

En la interfaz web (app.py):

```

Usuario sube imagen (JPG/PNG) desde file uploader
→ PIL.Image.open()
→ pil_a_bgr(): RGB numpy → BGR numpy
→ redimensionar_bgr(): reduce a máx. 640 px de ancho si supera ese tamaño
→ Motor(imagen_bgr)
→ motor.<método>(parámetros leídos de sliders)
→ resultado BGR numpy
→ bgr_a_rgb(): BGR numpy → RGB numpy
→ st.image(resultado_rgb, use_container_width=True)

```

En la interfaz de escritorio (main.py):

```

cv2.imread() carga imagen del dataset

```

- escalado a 400 x 340 px
- `Motor(imagen_bgr)`
- `motor.<método>(parámetros leídos de trackbars)`
- resultado BGR numpy
- `cv2.imshow()` en panel 2x4 con los 8 modos simultáneos

La diferencia fundamental: en el escritorio, todos los modos se calculan y muestran al mismo tiempo en un panel de 8 cuadrantes. En la web, solo el modo seleccionado se calcula, y el resultado aparece junto a la imagen original en dos columnas.

4.4 Cómo ejecutar el proyecto

Instalar dependencias:

```
pip install -r requirements.txt
```

Interfaz de escritorio (requiere dataset en images/):

```
python main.py
```

Navegación: A / D para cambiar imagen, S para guardar, Q para salir.

Interfaz web (no requiere dataset):

```
streamlit run app.py
```

Se abre en el navegador. Cargar una imagen desde el panel izquierdo y ajustar los sliders.

5. Pruebas y resultados

Probamos el proyecto con imágenes del dataset *Fruits Dataset (Images)* de Kaggle, que incluye carpetas separadas para manzanas, bananas, cerezas y otras frutas. Las capturas que presentamos a continuación corresponden a imágenes de manzanas rojas.

5.1 Interfaz web — pantalla inicial



Pantalla inicial de la interfaz web

La captura muestra el estado de la aplicación antes de cargar una imagen. Se observa el panel lateral con el uploader, el selector de técnica (Modo 1 – Blur / Brillo / Contraste seleccionado por defecto), y los tres sliders correspondientes ya visibles con sus valores iniciales (Blur=2, Brillo=0, Contraste=1.0). El área principal muestra el mensaje de instrucción. Esto confirma que la interfaz carga correctamente y muestra los controles específicos del modo activo incluso sin imagen.

5.2 Segmentación K-means HSV sobre manzana roja

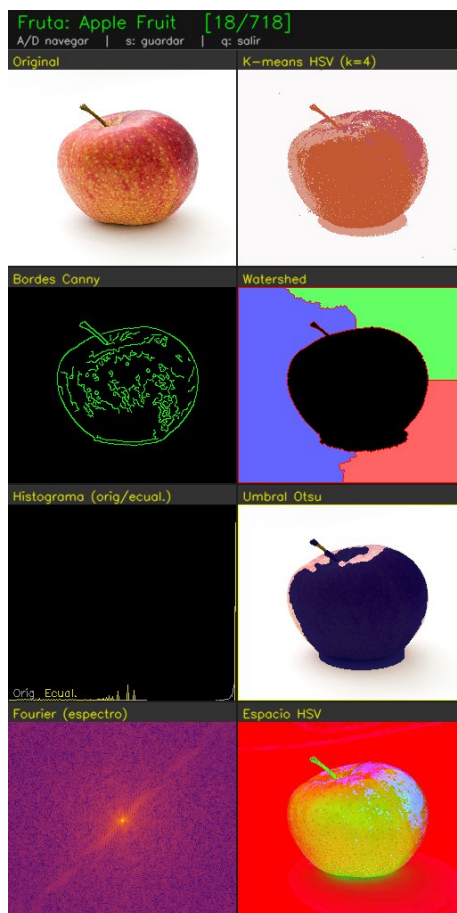


K-means con $k=4$ sobre manzana roja

En esta captura se aprecia el resultado del Modo 3 con $k=4$ sobre una imagen de manzana roja. En la imagen original se observa la manzana con gradientes de color natural: zonas más brillantes (reflejo), zonas más oscuras (sombra), la hoja verde y el fondo blanco. En la imagen procesada, K-means en espacio HSV identifica cuatro clusters: el cuerpo principal de la manzana queda reducido a un rojo uniforme, la hoja y el tallo agrupan en un tono verde-marrón, el reflejo especular forma un cluster propio (blanco apagado), y el fondo se asigna a un cluster cuarto (gris claro). La reducción de colores es clara y los límites entre regiones son abruptos, lo que evidencia tanto la fortaleza (separación limpia de regiones de color dominante) como la limitación (pérdida de gradientes sutiles dentro de cada región) de la técnica.

Limitación observada: con imágenes de manzanas que tienen zonas de transición suave entre rojo oscuro y rojo claro, K-means tiende a incluir ambas en el mismo cluster si son tonalmente similares en HSV, ignorando diferencias de saturación. Aumentar k a 5 o 6 resuelve esto pero genera fragmentación excesiva en el fondo.

5.3 Panel de escritorio — todos los modos simultáneos



Panel OpenCV con todos los modos activos

Esta captura del panel de escritorio muestra los ocho modos aplicados simultáneamente sobre la imagen 18/718 del conjunto de manzanas. Se observa:

- **Original:** manzana roja con frente que parece estar en un bol, sobre fondo oscuro.

- **K-means (k=4):** simplificación en cuatro regiones de color plano; se distingue la manzana del fondo con claridad.
- **Bordes Canny:** contornos verdes finos que delimitan la manzana y el recipiente. Los bordes internos de la textura de la piel son visibles, lo que indica que los umbrales permiten detectar variaciones sutiles de intensidad.
- **Watershed:** regiones coloreadas (azul, verde, rojo, negro) que separan la manzana de fondo. Se aprecia que el algoritmo identifica correctamente varios segmentos diferenciados, con bordes rojos en las fronteras.
- **Histograma (orig/ecual.):** curvas superpuestas que muestran la distribución de intensidad y su versión ecualizada. La curva ecualizada se extiende a lo largo de todo el rango, confirmando el efecto de mejora de contraste.
- **Umbral Otsu:** la máscara binaria con overlay azul delimita bien la manzana cuando el fondo es oscuro. El tinte azul en el fondo y el contorno amarillo son claramente visibles.
- **Fourier (espectro):** el espectro de frecuencias muestra el patrón característico con máximo de energía en el centro (bajas frecuencias de la imagen) y valores decrecientes hacia la periferia. La colorización con COLORMAP_INFERNO hace visibles diferencias que en escala de grises no serían perceptibles.
- **Espacio HSV:** la representación visual del espacio HSV muestra la manzana en colores falsos que revelan la distribución de tono y saturación; el rojo de la manzana aparece como un color saturado intenso, mientras que el fondo oscuro se muestra como zona casi negra en V.

Limitación observada en Watershed: cuando la fruta ocupa la mayor parte del frame y su fondo es muy oscuro, la transformada de distancia genera pocas semillas y tiende a tratar la manzana entera como un único segmento. Subir el umbral de distancia al 60–70 % del máximo mejora la separación, pero puede fragmentar la fruta en partes que no corresponden a objetos distintos.

Limitación general: la interfaz de escritorio calcula los ocho modos en cada frame para mostrarlos en el panel, lo que genera una carga computacional mayor que la interfaz web. En imágenes grandes, K-means y Watershed pueden introducir latencia perceptible al mover los sliders.

6. Mejoras futuras

Técnicas

- **Detección de objetos con YOLO o SSD:** la herramienta actual segmenta a nivel de píxel pero no identifica qué fruta es. Integrar un modelo preentrenado permitiría mostrar la etiqueta de la fruta junto con la segmentación.
- **Filtros de Fourier más elaborados:** actualmente solo implementamos pasa bajos circular. Un filtro pasa altos, pasa banda o notch permitiría explorar más fenómenos de frecuencia y sería útil para mejorar la detección de texturas.
- **Supresión de ruido con filtro bilateral:** a diferencia del gaussiano, el filtro bilateral suaviza preservando bordes, lo que mejoraría la calidad de entrada para Canny y Otsu en imágenes con ruido real.

- **Morfología expuesta al usuario:** el módulo `morfologia.py` implementa erosión, dilatación, apertura y cierre, pero ninguna interfaz los expone como modo propio. Un Modo 9 dedicado a morfología interactiva cubriría la Unidad 2 de manera más explícita.

Funcionales

- **Botón de descarga del resultado:** en la interfaz web, permitir descargar la imagen procesada con un botón (`st.download_button`) sin necesidad de capturas de pantalla.
- **Comparación deslizable (slider de opacidad):** en lugar de columnas lado a lado, una superposición ajustable con opacidad variable haría más visible la diferencia antes/después en regiones específicas.
- **Historial de procesamientos:** permitir encadenar múltiples técnicas sobre el mismo resultado (por ejemplo, `Blur` → `Canny` → `Watershed`) en lugar de aplicar cada una sobre la imagen original.
- **Soporte de video en tiempo real:** extender `main.py` para procesar la señal de una webcam frame a frame, lo que permitiría experimentar con objetos físicos reales en lugar de imágenes estáticas.

Visuales

- **Overlays informativos:** mostrar el valor del umbral calculado por Otsu, el número de segmentos detectados por Watershed, o el error de cuantización de K-means como texto sobre la imagen procesada.
- **Tema oscuro para la interfaz web:** Streamlit soporta tema personalizado en `config.toml`; un fondo oscuro haría las imágenes procesadas más legibles en el área principal.
- **Panel de métricas:** mostrar en la interfaz web métricas básicas de la imagen procesada (media de intensidad, contraste global, número de bordes detectados) para cuantificar el efecto de cada parámetro.